

Práctica 2: Análisis de Calidad Industrial (Industria 4.0)

Asignatura: Ciencia de Datos | Ingeniería en Sistemas Computacionales

1. Contexto Profesional

La empresa "TechManufacture" ha implementado sensores en sus líneas de ensamblaje. Se sospecha que las variaciones de temperatura y presión en las máquinas están afectando la calidad del producto final. Como Ingeniero de Datos, se te ha entregado un reporte de los últimos 500 lotes de producción para realizar un análisis estadístico exhaustivo.

2. Objetivo

Poner a prueba las habilidades de manipulación de datos con `dplyr`, cálculo de medidas estadísticas y visualización con `ggplot2` para generar interpretaciones de negocio.

3. Requerimientos

- Entorno: **RStudio**.
- Librerías: `tidyverse` (que incluye `dplyr` y `ggplot2`).
- Dataset: Se generará de forma sintética dentro del script.

4. Tareas a realizar

Fase A: Análisis Exploratorio (EDA)

1. Calcular la **media**, **mediana** y **desviación estándar** de la temperatura de operación.
2. Identificar si existen valores faltantes (`NA`) y aplicar una técnica de limpieza.
3. Determinar la **correlación de Pearson** entre la temperatura y la `tasa_error`.

Fase B: Agrupación y Segmentación

1. Comparar el rendimiento promedio (tasa de error) entre los turnos **Matutino** y **Vespertino**.
2. ¿Existe una diferencia significativa en la temperatura de operación entre turnos?

Fase C: Visualización e Interpretación

1. Crear un **Boxplot** para visualizar la dispersión de errores por turno (identificar *outliers*).

2. Generar un **Scatter Plot** (Gráfico de dispersión) con una línea de regresión para mostrar la relación entre temperatura y errores.

5. Entregable

Un archivo en formato **R Markdown (.Rmd)** o **Quarto (.qmd)** que incluya el código, las gráficas generadas y un párrafo de conclusión interpretando los resultados (¿Debería la empresa invertir en mejores sistemas de enfriamiento?).

```
# =====  
# PRÁCTICA 2: ANÁLISIS DE CALIDAD INDUSTRIAL  
# =====  
  
# Librerías  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Configuración  
np.random.seed(42)
```

```
# =====  
# Generación de datos sintéticos  
# =====  
  
n = 500  
  
temperatura = np.random.normal(80, 5, n)  
presion = np.random.normal(50, 8, n)  
turno = np.random.choice(["Matutino", "Vespertino"], n)  
  
# Relación entre temperatura y errores  
tasa_error = (temperatura * 0.15) + np.random.normal(0, 2, n)  
  
# Crear DataFrame  
df = pd.DataFrame({  
    'temperatura': temperatura,  
    'presion': presion,  
    'turno': turno,  
    'tasa_error': tasa_error  
})  
  
# Introducir algunos valores faltantes  
indices = np.random.choice(df.index, 20)  
df.loc[indices, 'temperatura'] = np.nan  
  
print("Primeras filas")  
print(df.head())
```

```

# =====
# FASE A: ANÁLISIS EXPLORATORIO
# =====

print("\nMedia:")
print(df['temperatura'].mean())

print("\nMediana:")
print(df['temperatura'].median())

print("\nDesviación estándar:")
print(df['temperatura'].std())

# Valores faltantes
print("\nValores faltantes:")
print(df.isnull().sum())

# Limpiar usando media
media_temp = df['temperatura'].mean()
df['temperatura'].fillna(media_temp,inplace=True)

print("\nValores faltantes después de limpieza:")
print(df.isnull().sum())

# Correlación Pearson
correlacion = df['temperatura'].corr(df['tasa_error'])

print("\nCorrelación temperatura vs tasa error:")
print(correlacion)

```

```

# =====
# FASE B: AGRUPACIÓN
# =====

promedio_turno = df.groupby("turno")["tasa_error"].mean()

print("\nPromedio de errores por turno")
print(promedio_turno)

temperatura_turno = df.groupby("turno")["temperatura"].mean()

print("\nTemperatura promedio por turno")
print(temperatura_turno)

```

```

# =====
# FASE C: VISUALIZACIÓN
# =====

# Boxplot

plt.figure(figsize=(8,5))

sns.boxplot(
    x='turno',
    y='tasa_error',
    data=df
)

plt.title("Errores por turno")
plt.show()

# Scatter con línea de regresión

plt.figure(figsize=(8,5))

sns.regplot(
    x='temperatura',
    y='tasa_error',
    data=df
)

plt.title("Temperatura vs Tasa de error")
plt.show()

```

Práctica 3: Análisis de Eficiencia en Logística Global

Asignatura: Ciencia de Datos | **Ingeniería en Sistemas Computacionales**

1. Contexto Profesional

La empresa "Global-Logistics ISC" utiliza dos modalidades de transporte para sus envíos nacionales: **Terrestre** y **Aéreo**. La gerencia de operaciones ha notado retrasos y sospecha que el tiempo de entrega no solo depende de la distancia, sino de ineficiencias en la modalidad de transporte bajo ciertas condiciones climáticas. Tu misión es analizar los datos de los últimos 600 envíos para determinar dónde están los cuellos de botella.

2. Objetivo

Validar de forma independiente la capacidad de limpiar datos, realizar análisis exploratorio, agrupar información y aplicar pruebas de hipótesis para la toma de decisiones basada en datos.

3. Requerimientos

- Entorno: **RStudio**.
- Librerías: `tidyverse` (`dplyr`, `ggplot2`).
- Dataset: Simulación de 600 registros generada en el script.

4. Tareas a realizar por el alumno

Fase 1: Calidad de Datos

1. **Limpieza:** El dataset contiene valores NA en la columna `distancia_km` debido a fallas en el GPS. Imputa estos valores usando la **mediana**.
2. **Descriptivos:** Calcula la media y desviación estándar del `tiempo_entrega_hrs`.

Fase 2: Análisis de Relaciones

1. **Correlación:** Calcula el coeficiente de Pearson entre `distancia_km` y `tiempo_entrega_hrs`. ¿Es la relación tan fuerte como se esperaba?

Fase 3: Comparativa de Modalidades

1. **Agrupación:** Agrupa por `tipo_transporte` y obtén el promedio de tiempo de entrega y el costo promedio.
2. **Prueba de Hipótesis:** Aplica un **t-test** para determinar si existe una diferencia significativa entre el tiempo de entrega del transporte **Terrestre** vs **Aéreo**.

Fase 4: Visualización

1. Genera un **Gráfico de Dispersión** que muestre la relación entre distancia y tiempo, diferenciando los puntos por modalidad de transporte.
2. Crea un **Boxplot** para comparar los tiempos de entrega entre modalidades e identificar valores atípicos (envíos extremadamente retrasados).

5. Entregable

Un reporte técnico en **R Markdown** donde se presente el análisis, las gráficas y una recomendación final: ¿Debería la empresa migrar más envíos al transporte aéreo para mejorar la satisfacción del cliente?

```
# =====  
# PRÁCTICA 3: ANÁLISIS DE EFICIENCIA  
# EN LOGÍSTICA GLOBAL  
# =====  
  
# Librerías  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
from scipy.stats import ttest_ind  
  
# Configuración  
np.random.seed(42)
```

```

# =====
# Generación de datos sintéticos
# =====

n = 600

tipo_transporte = np.random.choice(
    ["Terrestre", "Aereo"],
    n
)

distancia_km = np.random.normal(
    800,
    300,
    n
)

# Tiempo depende de distancia y transporte
tiempo_entrega_hrs = np.where(
    tipo_transporte=="Aereo",
    distancia_km*0.01 + np.random.normal(3,2,n),
    distancia_km*0.02 + np.random.normal(5,3,n)
)

costo = np.where(
    tipo_transporte=="Aereo",
    distancia_km*4 + np.random.normal(1000,200,n),
    distancia_km*2 + np.random.normal(500,100,n)
)

```



```

# Crear DataFrame

df = pd.DataFrame({
    "distancia_km": distancia_km,
    "tiempo_entrega_hrs": tiempo_entrega_hrs,
    "tipo_transporte": tipo_transporte,
    "costo": costo
})

# Introducir valores faltantes
indices = np.random.choice(df.index, 30)

df.loc[indices, 'distancia_km'] = np.nan

print("Primeras filas")
print(df.head())

```

```

# =====
# FASE 1: CALIDAD DE DATOS
# =====

print("\nValores faltantes")

print(df.isnull().sum())

# Imputar con mediana

mediana = df["distancia_km"].median()

df["distancia_km"].fillna(
    mediana,
    inplace=True
)

print("\nValores faltantes después")

print(df.isnull().sum())

```

```

# Imputar con mediana

mediana = df["distancia_km"].median()

df["distancia_km"].fillna(
    mediana,
    inplace=True
)

print("\nValores faltantes después")

print(df.isnull().sum())

# Estadísticos descriptivos

print("\nMedia tiempo entrega")

print(
    df["tiempo_entrega_hrs"].mean()
)

print("\nDesviación estándar")

print(
    df["tiempo_entrega_hrs"].std()
)

```

```

# =====
# FASE 2: RELACIÓN ENTRE VARIABLES
# =====

correlacion = df["distancia_km"].corr(
    df["tiempo_entrega_hrs"]
)

print("\nCorrelación:")

print(correlacion)

```

```

# =====
# FASE 3: COMPARATIVA
# =====

agrupado = df.groupby(
    "tipo_transporte"
)[["tiempo_entrega_hrs", "costo"]].mean()

print("\nPromedios por transporte")

print(agrupado)

```

```

# T-Test

terrestre = df[
    df["tipo_transporte"]=="Terrestre"
][["tiempo_entrega_hrs"]]

aereo = df[
    df["tipo_transporte"]=="Aereo"
][["tiempo_entrega_hrs"]]

t,p = ttest_ind(
    terrestre,
    aereo
)

print("\nT-statistic:",t)
print("P-value:",p)

if p<0.05:
    print("Existe diferencia significativa")
else:
    print("No existe diferencia significativa")

```

```
# =====
# FASE 4: VISUALIZACIÓN
# =====

# Scatter

plt.figure(figsize=(9,5))

sns.scatterplot(
    x='distancia_km',
    y='tiempo_entrega_hrs',
    hue='tipo_transporte',
    data=df
)

plt.title(
    "Distancia vs Tiempo"
)

plt.show()
```

```
# Boxplot

plt.figure(figsize=(8,5))

sns.boxplot(
    x='tipo_transporte',
    y='tiempo_entrega_hrs',
    data=df
)

plt.title(
    "Tiempo de entrega por transporte"
)

plt.show()
```

Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial

1. Contexto Profesional

La planta de fundición "Metal-Tech" ha instalado 12 sensores en una caldera crítica. Los sensores miden temperatura, presión, vibración, flujo de gas, emisiones de CO₂, entre otros. El equipo de TI reporta que procesar las 12 señales en tiempo real satura el ancho de banda del PLC. Tu tarea es aplicar **PCA** para reducir estas 12 variables a los componentes principales que capturen al menos el **85% de la varianza**, permitiendo un monitoreo eficiente.

2. Objetivo

Aplicar el algoritmo PCA en R para identificar variables redundantes y reducir la dimensionalidad de un sistema de sensores industriales.

3. Requerimientos

- Entorno: **RStudio**.
- Librería: `tidyverse`, `factoextra` (opcional para mejores gráficos).

4. Instrucciones para el Estudiante

Fase 1: Preparación

1. Cargar el dataset de sensores (generado en el script).
2. Observar la correlación inicial entre sensores. ¿Hay sensores que miden "lo mismo"?

Fase 2: Ejecución de PCA

1. Utilizar la función `prcomp()`. **Importante:** Asegúrate de estandarizar los datos (`scale. = TRUE`).
2. Analizar el objeto resultante con `summary()`.

Fase 3: Selección de Componentes

1. Generar el **Scree Plot**.
2. Determinar cuántos componentes son necesarios para alcanzar el 85% de varianza acumulada.
3. Identificar qué sensores originales tienen más peso ("loadings") en el primer componente principal.

Fase 4: Visualización

1. Crear un **Biplot** para visualizar cómo se agrupan los lotes de producción en el nuevo espacio de dimensiones reducidas.

```
2. # =====
3. # PRÁCTICA 4: REDUCCIÓN DE DIMENSIONALIDAD
4. # EN MONITOREO INDUSTRIAL (PCA)
5. # =====
6.
7. # Librerías
8.
9. import numpy as np
10. import pandas as pd
11. import matplotlib.pyplot as plt
12. import seaborn as sns
13.
14. from sklearn.preprocessing import StandardScaler
15. from sklearn.decomposition import PCA
16.
17. # Configuración
18.
19. np.random.seed(42)
20.
21. # =====
22. # Generación del dataset
23. # =====
24.
25. n=200
26.
27. temp1=np.random.normal(100,5,n)
28.
29. temp2=temp1+np.random.normal(0,1,n)
30.
31. presion1=np.random.normal(50,10,n)
32.
33. vibracion=(temp1*0.5)+np.random.normal(0,2,n)
34.
35. datos_sensores=pd.DataFrame({
36.
37. 'temp_nucleo':temp1,
38.
39. 'temp_escape':temp2,
40.
41. 'presion_interna':presion1,
42.
43. 'vibracion_motor':vibracion,
44.
```

```

45. 'flujo_gas': np.random.normal(20,2,n),
46.
47. 'oxigeno': np.random.normal(15,1,n),
48.
49. 'co2': temp1*0.2+np.random.normal(0,0.5,n),
50.
51. 'humedad': np.random.uniform(30,40,n),
52.
53. 'voltaje': np.random.normal(220,2,n),
54.
55. 'corriente': np.random.normal(15,0.5,n),
56.
57. 'ruido_db': vibracion*1.2+np.random.normal(0,1,n),
58.
59. 'eficiencia': 100-(temp1*0.1)
60.
61. })
62.
63. print(datos_sensores.head())
64.
65. # =====
66. # FASE 1: MATRIZ DE CORRELACIÓN
67. # =====
68.
69. plt.figure(figsize=(12,8))
70.
71. sns.heatmap(
72.     datos_sensores.corr(),
73.     annot=True,
74.     cmap="coolwarm"
75.)
76.
77. plt.title(
78.     "Matriz de correlación"
79.)
80.
81. plt.show()
82.
83. # =====
84. # FASE 2: ESTANDARIZACIÓN
85. # =====
86.
87. escalador=StandardScaler()
88.
89. datos_escalados=escalador.fit_transform(

```

```

90.     datos_sensores
91.)
92.
93.# =====
94.# FASE 3: EJECUCIÓN PCA
95.# =====
96.
97.pca=PCA()
98.
99.componentes=pca.fit_transform(
100.     datos_escalados
101. )
102.
103.     # Varianza explicada
104.
105.     varianza=pca.explained_variance_ratio_
106.
107.     varianza_acumulada=np.cumsum(
108.         varianza
109.     )
110.
111.     print("\nVarianza explicada:")
112.
113.     print(varianza)
114.
115.     print("\nVarianza acumulada:")
116.
117.     print(varianza_acumulada)
118.
119.     # =====
120.     # SCREE PLOT
121.     # =====
122.
123.     plt.figure(figsize=(8,5))
124.
125.     plt.plot(
126.         range(1,len(varianza)+1),
127.         varianza_acumulada
128.     )
129.
130.     plt.axhline(
131.         y=0.85,
132.         linestyle="--"
133.     )
134.

```



```

135.     plt.xlabel(
136.         "Número de componentes"
137.     )
138.
139.     plt.ylabel(
140.         "Varianza acumulada"
141.     )
142.
143.     plt.title(
144.         "Scree Plot"
145.     )
146.
147.     plt.show()
148.
149.     # =====
150.     # COMPONENTES NECESARIOS PARA 85%
151.     # =====
152.
153.     componentes_85=np.argmax(
154.         varianza_acumulada>=0.85
155.     )+1
156.
157.     print("\nComponentes necesarios:")
158.
159.     print(componentes_85)
160.
161.     # =====
162.     # LOADINGS
163.     # =====
164.
165.     loadings=pd.DataFrame(
166.
167.         pca.components_.T,
168.
169.         columns=[
170.             f'PC{i+1}'
171.             for i in range(
172.                 len(datos_sensores.columns)
173.             )
174.         ],
175.
176.         index=datos_sensores.columns
177.     )
178.
179.

```

```
180.     print("\nLoadings PC1 y PC2")
181.
182.     print(
183.         loadings[['PC1', 'PC2']]
184.     )
185.
186.     # =====
187.     # BIPLLOT
188.     # =====
189.
190.     plt.figure(figsize=(10,6))
191.
192.     plt.scatter(
193.         componentes[:,0],
194.         componentes[:,1]
195.     )
196.
197.     plt.xlabel("PC1")
198.
199.     plt.ylabel("PC2")
200.
201.     plt.title(
202.         "Biplot PCA"
203.     )
204.
205.     plt.show()
```


Reporte Técnico: Reducción de Dimensionalidad en Monitoreo Industrial

Unidad 2: Tratamiento de Datos - Ciencia de Datos"

1. Introducción y Objetivo

Instrucciones: Describe brevemente el problema de la planta "Metal-Tech". ¿Por qué es necesario reducir las dimensiones de los 12 sensores? ¿Qué beneficios técnicos y económicos aporta el PCA en este escenario?

2. Metodología y Carga de Datos

En esta sección se inicializa el entorno y se cargan las señales de los sensores.

```
# Generación del dataset de sensores (Código proporcionado en la
práctica)
set.seed(42)
n <- 200
temp1 <- rnorm(n, 100, 5)
temp2 <- temp1 + rnorm(n, 0, 1)
presion1 <- rnorm(n, 50, 10)
vibracion <- (temp1 * 0.5) + rnorm(n, 0, 2)

datos_sensores <- data.frame(
  temp_nucleo = temp1, temp_escape = temp2, presion_interna = presion1,
  vibracion_motor = vibracion, flujo_gas = rnorm(n, 20, 2),
  oxigeno = rnorm(n, 15, 1), co2 = temp1 * 0.2 + rnorm(n, 0, 0.5),
  humedad = runif(n, 30, 40), voltaje = rnorm(n, 220, 2),
  corriente = rnorm(n, 15, 0.5), ruido_db = vibracion * 1.2 + rnorm(n, 0,
1),
  eficiencia = 100 - (temp1 * 0.1)
)

head(datos_sensores)
```

3. Ejecución del Algoritmo PCA

Se aplica la función `prcomp` asegurando la estandarización de las variables.

```
# Ejecución del PCA con escalamiento
pca_modelo <- prcomp(datos_sensores, center = TRUE, scale. = TRUE)

# Resumen de varianza
summary(pca_modelo)
```

4. Análisis de Resultados

4.1 Selección de Componentes (Scree Plot)

Tarea: Genera la gráfica de sedimentación e identifica el "codo". Justifica cuántos componentes seleccionarías para cumplir con el objetivo del 85% de varianza.

```
# Código para generar el gráfico de sedimentación
plot(pca_modelo, type = "l", main = "Gráfico de Sedimentación (Scree Plot)")
abline(h = 1, col="red", lty=2) # Criterio de Kaiser
```

Análisis del estudiante: ¿Cuántos componentes explican el 85%? ¿Por qué elegiste ese número?

4.2 Cargas de los Componentes (Loadings)

Tarea: Observa qué variables originales tienen más peso en los primeros dos componentes.

```
# Ver la rotación (loadings)
round(pca_modelo$rotation[, 1:2], 3)
```

Interpretación: ¿Qué representan físicamente el PC1 y el PC2? (Ejemplo: PC1 parece representar el 'Estado Térmico' del sistema).

4.3 Visualización del Biplot

```
# Visualización de variables y observaciones
biplot(pca_modelo, cex = 0.7, main = "Biplot de Sensores Industriales")
```

5. Conclusiones y Recomendaciones

Instrucciones: Basado en tu análisis, responde a la gerencia de TI:

1. ¿A cuántas variables podemos reducir el monitoreo sin perder información crítica?
2. ¿Cuáles sensores son redundantes y podrían dejar de enviarse por el bus de datos?
3. ¿Cómo impacta esta reducción en la eficiencia del sistema de monitoreo en tiempo real?

Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

Unidad 2: Tratamiento de Datos – TecNM

Contexto Profesional

Eres el líder de seguridad en un Centro de Datos. Tienes sensores que capturan 10 métricas distintas por cada conexión. Analizar 10 dimensiones en tiempo real es costoso. Tu objetivo es encontrar los "Componentes Principales" que describan el estado de la red (Normal vs. Anomalía) reduciendo la carga computacional.

Paso 1: Generación y Exploración de Datos

Primero, simulamos los datos de red. Observa que algunas variables están muy relacionadas (ej. bytes y paquetes).

```
set.seed(123)
n <- 300
# Datos simulados de tráfico
datos_red <- data.frame(
  duracion_ms = rnorm(n, 50, 10),
  paquetes_enviados = rnorm(n, 100, 20),
  bytes_enviados = NA, # Se calculará con correlación
  errores_checksum = rpois(n, 2),
  reintentos_tcp = NA,
  latencia_avg = rnorm(n, 15, 5),
  jitter = rnorm(n, 2, 0.5),
  carga_cpu_router = NA,
  uso_memoria_sw = rnorm(n, 40, 10),
  peticiones_http = rnorm(n, 200, 50)
)

# Creamos dependencias (Redundancia)
datos_red$bytes_enviados <- datos_red$paquetes_enviados * 1500 + rnorm(n,
0, 500)
datos_red$reintentos_tcp <- datos_red$errores_checksum * 1.5 + rnorm(n,
0, 0.5)
datos_red$carga_cpu_router <- (datos_red$paquetes_enviados * 0.4) +
(datos_red$latencia_avg * 0.2)

head(datos_red)
```

Interpretación Docente: Aquí el alumno debe notar que `bytes_enviados` es casi un espejo de `paquetes_enviados`. En ingeniería, esto es **redundancia**. El PCA detectará que no necesitamos ambas columnas para entender el flujo.

Paso 2: Estandarización (Scaling)

El PCA es sensible a las magnitudes. Si no escalamos, la variable `bytes_enviados` (que está en miles) dominará el análisis sobre `jitter` (que está en unidades).

```
# Ejecutamos PCA con scale. = TRUE
pca_red <- prcomp(datos_red, center = TRUE, scale. = TRUE)
```

Interpretación Docente: Escalar transforma los datos para que todos tengan media 0 y desviación 1. Esto garantiza que cada métrica de red tenga el mismo "voto" inicial en el algoritmo.

Paso 3: Análisis de Varianza (Varianza Explicada)

¿Cuánta información logramos retener?

```
summary(pca_red)
plot(pca_red, type = "l", main = "Scree Plot: Tráfico de Red")
```

Interpretación Docente: El alumno debe mirar la fila "Cumulative Proportion".

- **Punto clave:** Si el PC1 y PC2 suman > 0.70 (70%), significa que con solo 2 nuevas variables podemos explicar la mayor parte de lo que pasa en la red. El "codo" en la gráfica nos dice gráficamente dónde parar.

Paso 4: Interpretación de Cargas (Loadings)

¿Qué significan los nuevos componentes?

```
# Ver pesos de las variables originales en los componentes
round(pca_red$rotation[, 1:2], 3)
```

Interpretación Docente: > * Si el **PC1** tiene valores altos en `paquetes`, `bytes` y `carga_cpu`, el alumno interpreta que **PC1 = Volumen de Tráfico**.

- Si el **PC2** tiene valores altos en `latencia` y `jitter`, el alumno interpreta que **PC2 = Calidad/Estabilidad de Conexión**.
- ¡Hemos convertido 10 variables en 2 conceptos de ingeniería claros!

Paso 5: Visualización (Biplot)

El diagnóstico final.

```
biplot(pca_red, cex = 0.8, main = "Mapa de Estado de Red")
```

Interpretación Docente: > * **Flechas paralelas:** Indican variables redundantes (ej. `bytes` y `paquetes`).

- **Flechas largas:** Son las variables que más influyen en el sistema.
- **Puntos aislados:** Son conexiones anómalas (posibles ataques o fallas) que se alejan del comportamiento normal de la red.

```

• # =====
• # PRÁCTICA 5: REDUCCIÓN DE DIMENSIONES
• # EN TRÁFICO DE RED (PCA)
• # =====
•
• # Librerías
•
• import numpy as np
• import pandas as pd
• import matplotlib.pyplot as plt
• import seaborn as sns
•
• from sklearn.preprocessing import StandardScaler
• from sklearn.decomposition import PCA
•
• # Configuración
•
• np.random.seed(123)
•
• # =====
• # PASO 1: GENERACIÓN DE DATOS
• # =====
•
• n=300
•
• datos_red=pd.DataFrame({
•
•     'duracion_ms':np.random.normal(50,10,n),
•
•     'paquetes_enviados':np.random.normal(100,20,n),
•
•     'errores_checksum':np.random.poisson(2,n),
•
•     'latencia_avg':np.random.normal(15,5,n),
•
•     'jitter':np.random.normal(2,0.5,n),
•
•     'uso_memoria_sw':np.random.normal(40,10,n),
•
•     'peticiones_http':np.random.normal(200,50,n)
•
• })

```



```

•
• # Dependencias (redundancia)
•
•
• datos_red['bytes_enviados']=(
• datos_red['paquetes_enviados']*1500
• +
• np.random.normal(0,500,n)
• )
•
•
• datos_red['reintentos_tcp']=(
• datos_red['errores_checksum']*1.5
• +
• np.random.normal(0,0.5,n)
• )
•
•
• datos_red['carga_cpu_router']=(
• (datos_red['paquetes_enviados']*0.4)
• +
• (datos_red['latencia_avg']*0.2)
• )
•
•
• print(datos_red.head())
•
•
• # =====
• # PASO 2: MATRIZ DE CORRELACIÓN
• # =====
•
•
• plt.figure(figsize=(12,8))
•
•
• sns.heatmap(
•     datos_red.corr(),
•     annot=True,
•     cmap="coolwarm"
• )
•
•
• plt.title(
•     "Correlación entre métricas de red"
• )
•
•
• plt.show()
•
•
• # =====
• # PASO 3: ESTANDARIZACIÓN
• # =====
•

```

```

• escalador=StandardScaler()
•
• datos_escalados=escalador.fit_transform(
•     datos_red
• )
•
• # =====
• # PASO 4: PCA
• # =====
•
• pca=PCA()
•
• componentes=pca.fit_transform(
•     datos_escalados
• )
•
• varianza=pca.explained_variance_ratio_
•
• varianza_acumulada=np.cumsum(
•     varianza
• )
•
• print("\nVarianza explicada")
•
• print(varianza)
•
• print("\nVarianza acumulada")
•
• print(varianza_acumulada)
•
• # =====
• # SCREE PLOT
• # =====
•
• plt.figure(figsize=(8,5))
•
• plt.plot(
•     range(
•         1,
•         len(varianza)+1
•     ),
•     varianza_acumulada
• )
•
• plt.xlabel(

```

```

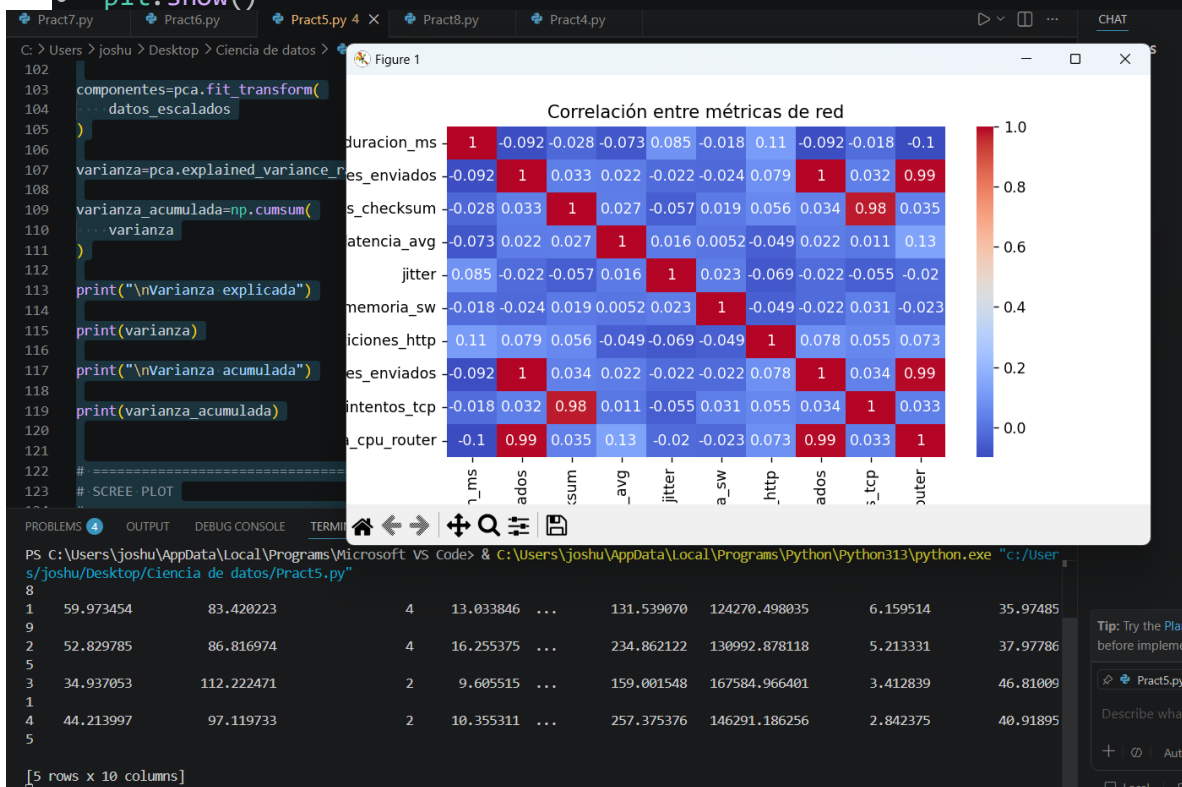
• "Componentes"
• )
•
• plt.ylabel(
• "Varianza acumulada"
• )
•
• plt.title(
• "Scree Plot: Tráfico de Red"
• )
•
• plt.axhline(
• y=0.70,
• linestyle='--'
• )
•
• plt.show()
•
• # =====
• # LOADINGS
• # =====
•
• loadings=pd.DataFrame(
•
• pca.components_.T,
•
• columns=[
• f'PC{i+1}'
• for i in range(
• len(datos_red.columns)
• )
• ],
•
• index=datos_red.columns
•
• )
•
• print("\nPesos PC1 y PC2")
•
• print(
• loadings[['PC1', 'PC2']]
• )
•
• # =====
• # BILOT

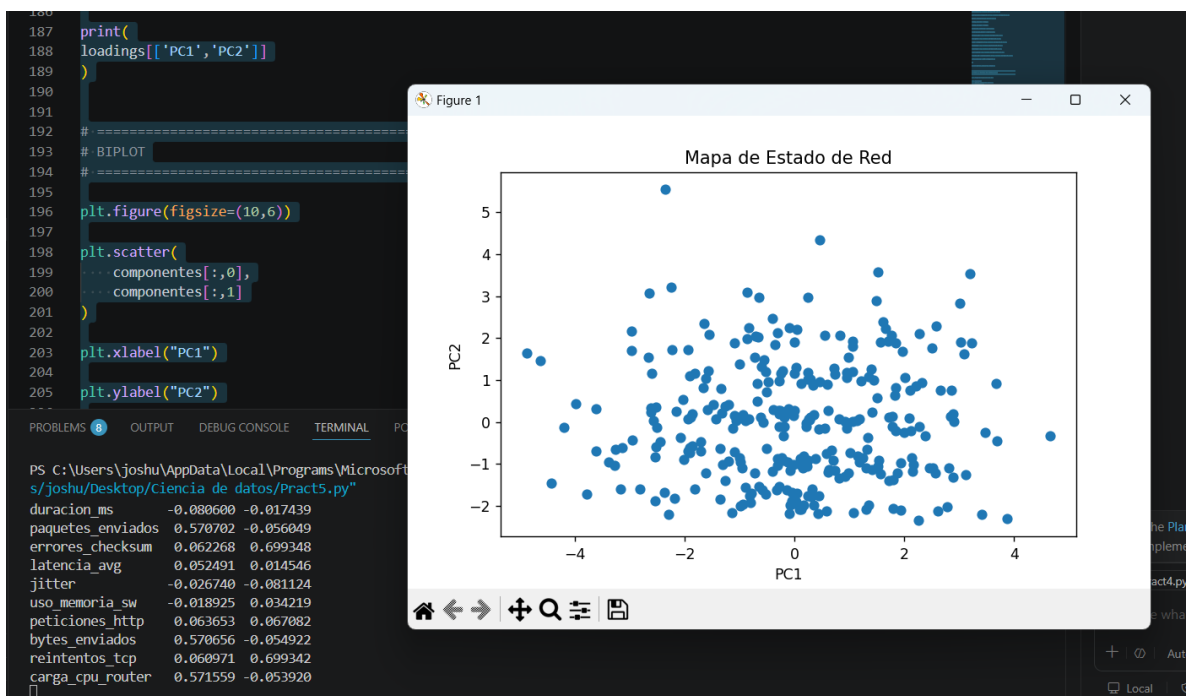
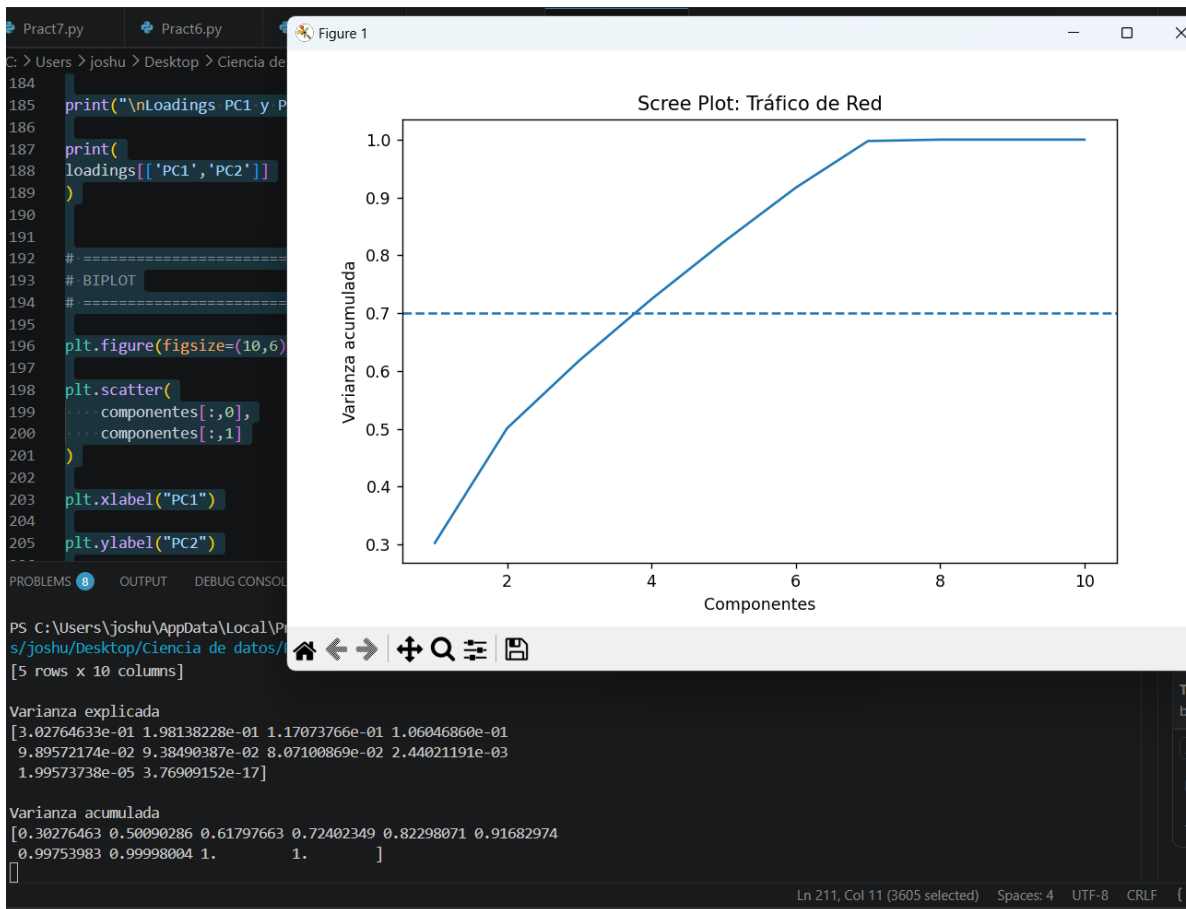
```

```

• # =====
•
• plt.figure(figsize=(10,6))
•
• plt.scatter(
• componentes[:,0],
• componentes[:,1]
• )
•
• plt.xlabel(
• 'PC1'
• )
•
• plt.ylabel(
• 'PC2'
• )
•
• plt.title(
• 'Mapa de Estado de Red'
• )
•
• plt.show()

```





title: "Práctica 6: Optimización de Sensores en Smart Cities" subtitle: "Unidad 2: Tratamiento de Datos - TecNM" author: "Guía Paso a Paso con Interpretación" output: html_document: toc: true theme: united

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
```

Contexto Profesional

Imagina que trabajas para el departamento de Innovación Tecnológica de una metrópoli. Han instalado 10 sensores en cada intersección principal para medir: calidad del aire, ruido, tráfico y clima. Transmitir las 10 señales por cada nodo de la ciudad satura la red de bajo consumo (LoRaWAN). Tu misión es aplicar **PCA** para reducir estas métricas a los componentes esenciales que describan el "Estado de la Ciudad".

Paso 1: Generación de Datos Urbanos

Simulamos el comportamiento de los sensores. Nota cómo variables como el CO2 y la densidad de vehículos suelen moverse juntas.

```
set.seed(456)
n <- 400
# Datos base de sensores urbanos
datos_urbanos <- data.frame(
  temp_ambiente = rnorm(n, 28, 4),
  humedad_rel = rnorm(n, 60, 10),
  co2_ppm = NA, # Relacionado con tráfico
  no2_ppb = NA, # Relacionado con tráfico
  particulas_pm25 = NA, # Relacionado con CO2
  nivel_ruido_db = NA, # Relacionado con vehículos
  densidad_vehiculos = rnorm(n, 120, 30),
  velocidad_viento = rnorm(n, 15, 5),
  radiacion_solar = rnorm(n, 800, 100),
  conteo_peatones = rnorm(n, 50, 15)
)

# Creación de Redundancia y Correlación (Lógica de ciudad)
```

```

datos_urbanos$co2_ppm <- (datos_urbanos$densidad_vehiculos * 3.5) +
rnorm(n, 300, 50)
datos_urbanos$no2_ppb <- (datos_urbanos$co2_ppm * 0.1) + rnorm(n, 5, 2)
datos_urbanos$particulas_pm25 <- (datos_urbanos$co2_ppm * 0.05) +
rnorm(n, 10, 3)
datos_urbanos$nivel_ruido_db <- (datos_urbanos$densidad_vehiculos * 0.2)
+ 50 + rnorm(n, 0, 5)

head(datos_urbanos)

```

Interpretación Docente: El estudiante debe observar que el CO₂, el NO₂ y las partículas PM_{2.5} están fuertemente ligados a la densidad_vehiculos. En lugar de monitorear 4 sensores que dicen casi lo mismo, el PCA nos dirá que podemos crear un solo "Índice de Contaminación Vehicular".

Paso 2: Estandarización de las Métricas

¿Por qué es obligatorio el comando `scale. = TRUE`? Porque el CO₂ se mide en cientos de unidades, mientras que la humedad en porcentajes.

```

# Ejecución del PCA con escalamiento centrado
pca_ciudad <- prcomp(datos_urbanos, center = TRUE, scale. = TRUE)

```

Interpretación Docente: Sin estandarización, las variables con rangos numéricos más grandes "secuestrarían" el análisis. El escalamiento permite que el sensor de temperatura tenga el mismo peso inicial que el de CO₂ para detectar cambios en el patrón de la ciudad.

Paso 3: Análisis de Varianza Acumulada

Determinamos cuánta información "sobrevive" a la reducción.

```

# Resumen de varianza por componente
summary(pca_ciudad)

# Visualización del Scree Plot
plot(pca_ciudad, type = "l",
     main = "Scree Plot: Sensores Smart City",
     ylab = "Varianza Explicada", xlab = "Componentes")
abline(h = 1, col="red", lty=2)

```

Interpretación Docente: > * **Varianza Proporcional:** Si el PC1 explica el 55% de la varianza, significa que más de la mitad de la complejidad de la ciudad se resume en una sola dimensión.

- **Criterio de Selección:** Los alumnos deben buscar el punto donde la curva se aplanan. Si los primeros 3 componentes explican el 85%, el ahorro de transmisión de datos para la ciudad es del **70%**.

Paso 4: Interpretación de los Componentes (Loadings)

Traducción de matemáticas a ingeniería urbana.

```
# Pesos de las variables originales
round(pca_ciudad$rotation[, 1:2], 3)
```

Interpretación Docente: El alumno debe realizar la siguiente deducción:

- **PC1:** Si tiene cargas altas en vehiculos, co2, no2 y ruido, el **PC1 = Actividad e Impacto Vehicular.**
- **PC2:** Si tiene cargas altas en temp, humedad y radiacion, el **PC2 = Factor Climático/Meteorológico.**
- Hemos simplificado el tablero de control de la ciudad de 10 indicadores a solo 2 macro-indicadores.

Paso 5: El Biplot de Diagnóstico Urbano

Visualizamos el estado de la metrópoli.

```
biplot(pca_ciudad, cex = 0.7, main = "Mapa Biplot de Estado Urbano")
```

Interpretación Docente: > * **Flechas paralelas:** Indican sensores que se pueden consolidar. Por ejemplo, si las flechas de paquetes_pm25 y no2_ppb apuntan al mismo lugar, son redundantes.

- **Ángulo de 90 grados:** Si la flecha de viento es perpendicular a la de vehiculos, el viento no depende del tráfico.
- **Puntos atípicos:** Aquellos puntos muy alejados en la dirección de la flecha de ruido y co2 representan horas pico o incidentes de tráfico severo detectados por el sistema.

```
• # =====
• # PRÁCTICA 6: OPTIMIZACIÓN DE SENSORES
• # EN SMART CITIES (PCA)
• # =====
•
• # Librerías
•
• import numpy as np
• import pandas as pd
• import matplotlib.pyplot as plt
• import seaborn as sns
•
• from sklearn.preprocessing import StandardScaler
• from sklearn.decomposition import PCA
```



```

•
• # Configuración
•
• np.random.seed(456)
•
• # =====
• # PASO 1: GENERACIÓN DE DATOS
• # =====
•
• n=400
•
• datos_urbanos=pd.DataFrame({
•
• 'temp_ambiente':np.random.normal(28,4,n),
•
• 'humedad_rel':np.random.normal(60,10,n),
•
• 'densidad_vehiculos':np.random.normal(120,30,n),
•
• 'velocidad_viento':np.random.normal(15,5,n),
•
• 'radiacion_solar':np.random.normal(800,100,n),
•
• 'conteo_peatones':np.random.normal(50,15,n)
•
• })
•
• # Crear correlaciones y redundancia
•
• datos_urbanos['co2_ppm']=(
• datos_urbanos['densidad_vehiculos']*3.5
• +
• np.random.normal(300,50,n)
• )
•
• datos_urbanos['no2_ppb']=(
• datos_urbanos['co2_ppm']*0.1
• +
• np.random.normal(5,2,n)
• )
•
• datos_urbanos['particulas_pm25']=(
• datos_urbanos['co2_ppm']*0.05
• +
• np.random.normal(10,3,n)

```

```

• )
•
• datos_urbanos['nivel_ruido_db']=(
• datos_urbanos['densidad_vehiculos']*0.2
• +
• 50
• +
• np.random.normal(0,5,n)
• )
•
• print(datos_urbanos.head())
•
• # =====
• # PASO 2: MATRIZ DE CORRELACIÓN
• # =====
•
• plt.figure(figsize=(12,8))
•
• sns.heatmap(
•     datos_urbanos.corr(),
•     annot=True,
•     cmap='coolwarm'
• )
•
• plt.title(
•     "Correlación de sensores urbanos"
• )
•
• plt.show()
•
• # =====
• # PASO 3: ESTANDARIZACIÓN
• # =====
•
• escalador=StandardScaler()
•
• datos_escalados=escalador.fit_transform(
•     datos_urbanos
• )
•
• # =====
• # PASO 4: PCA
• # =====
•
• pca=PCA()

```

```

•
• componentes=pca.fit_transform(
•     datos_escalados
• )
•
• varianza=pca.explained_variance_ratio_
•
• varianza_acumulada=np.cumsum(
•     varianza
• )
•
• print("\nVarianza explicada")
•
• print(varianza)
•
• print("\nVarianza acumulada")
•
• print(varianza_acumulada)
•
• # =====
• # SCREE PLOT
• # =====
•
• plt.figure(figsize=(8,5))
•
• plt.plot(
•     range(
•         1,
•         len(varianza)+1
•     ),
•     varianza_acumulada
• )
•
• plt.xlabel(
•     "Componentes"
• )
•
• plt.ylabel(
•     "Varianza acumulada"
• )
•
• plt.title(
•     "Scree Plot: Smart City"
• )
•

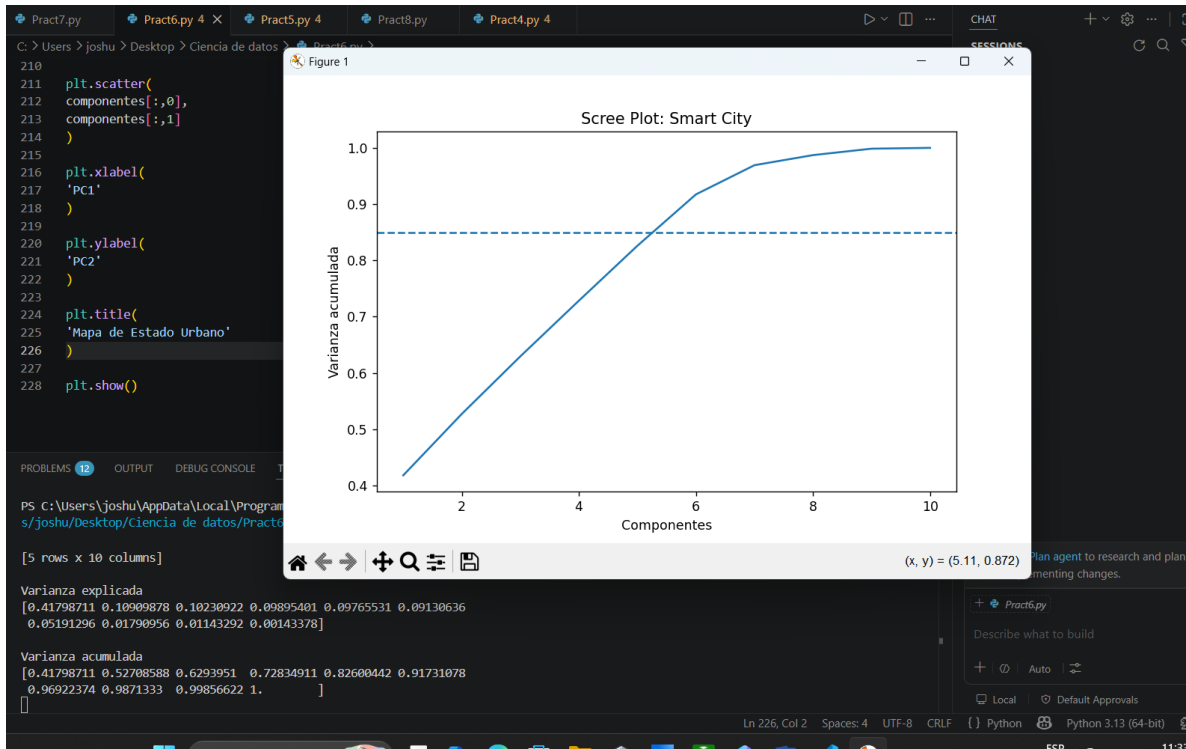
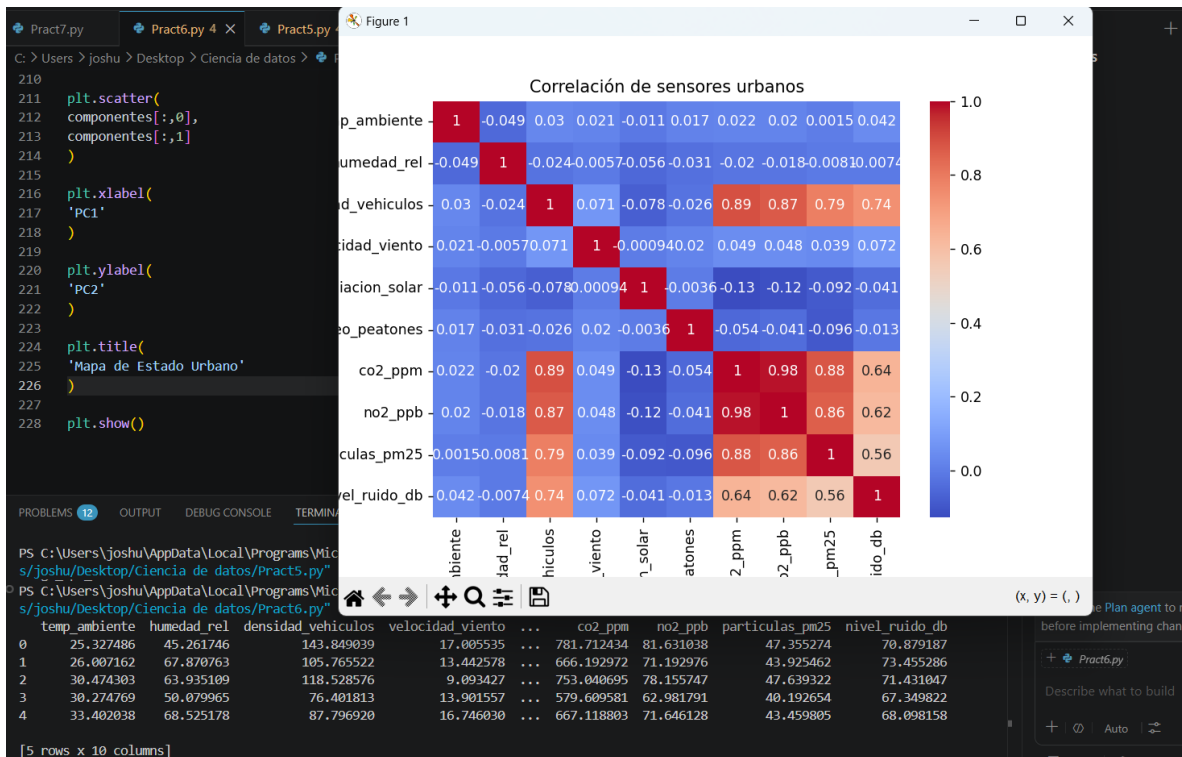
```

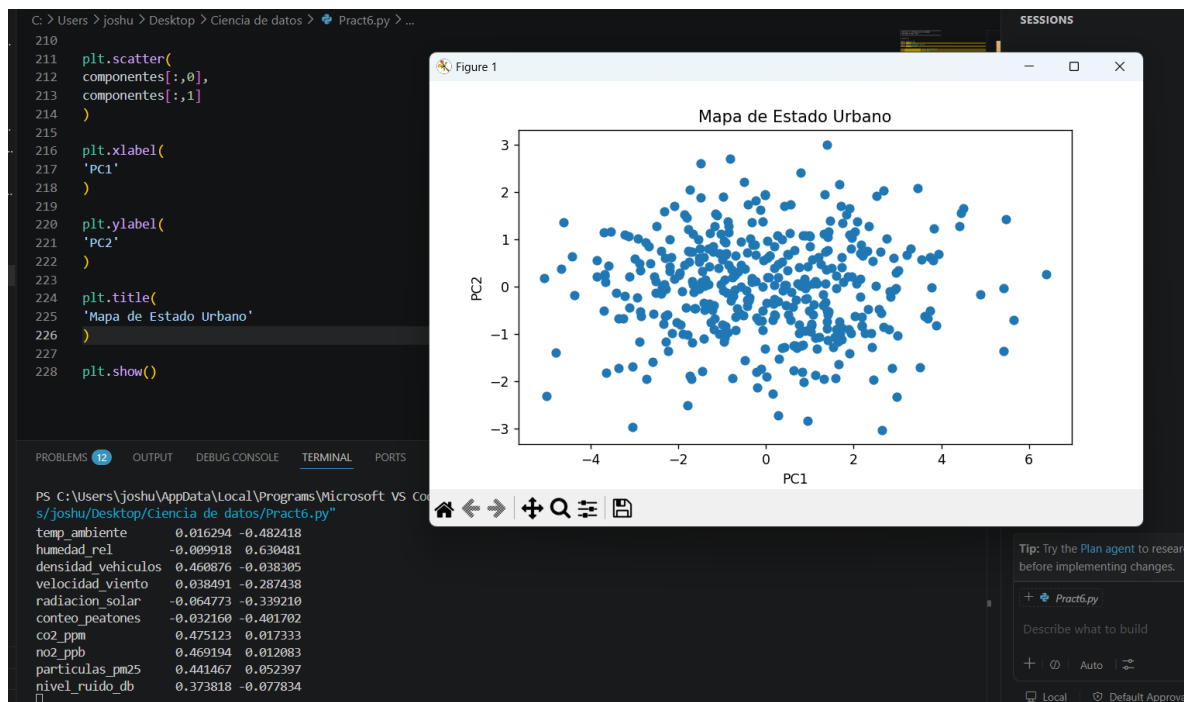
```

• plt.axhline(
• y=0.85,
• linestyle='--'
• )
•
• plt.show()
•
• # =====
• # CANTIDAD DE COMPONENTES NECESARIOS
• # =====
•
• componentes_85=np.argmax(
• varianza_acumulada>=0.85
• )+1
•
• print(
• "\nComponentes necesarios:"
• )
•
• print(
• componentes_85
• )
•
• # =====
• # LOADINGS
• # =====
•
• loadings=pd.DataFrame(
•
• pca.components_.T,
•
• columns=[
• f'PC{i+1}'
• for i in range(
• len(datos_urbanos.columns)
• )
• ],
•
• index=datos_urbanos.columns
•
• )
•
• print("\nPesos PC1 y PC2")
•
• print(

```

```
• loadings[['PC1','PC2']]
• )
•
• # =====
• # BIPLLOT
• # =====
•
• plt.figure(figsize=(10,6))
•
• plt.scatter(
• componentes[:,0],
• componentes[:,1]
• )
•
• plt.xlabel(
• 'PC1'
• )
•
• plt.ylabel(
• 'PC2'
• )
•
• plt.title(
• 'Mapa de Estado Urbano'
• )
•
• plt.show()
```





Práctica 7: Implementación de Modelos de Machine Learning

Unidad 3: Modelado - Ciencia de Datos (TecNM)

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
library(caret) # Librería estándar para entrenamiento de modelos
library(glmnet) # Para modelos Lasso
```

Contexto Profesional

La empresa "DataSystems S.A." desea automatizar su departamento de Recursos Humanos. Necesitan predecir:

1. **Regresión:** El salario esperado de un nuevo perfil tecnológico.
2. **Clasificación:** Si un candidato permanecerá en la empresa más de 2 años (Retención).

3. **Clustering:** Agrupar empleados actuales para planes de capacitación personalizados.

Paso 1: Preparación del Dataset

Generamos datos que incluyan experiencia, nivel de certificaciones, habilidades sociales y salario.

```
set.seed(789)
n <- 500
df_empleados <- data.frame(
  experiencia = rnorm(n, 5, 2),
  certificaciones = rpois(n, 3),
  habilidades_sociales = runif(n, 1, 10),
  remoto = sample(c(0, 1), n, replace = TRUE)
)

# Variable objetivo continua (Salario)
df_empleados$salario <- 25000 + (df_empleados$experiencia * 5000) +
  (df_empleados$certificaciones * 2000) + rnorm(n,
0, 3000)

# Variable objetivo binaria (Retención: 1=Se queda, 0=Se va)
# Probabilidad depende de salario y habilidades sociales
prob <- plogis(-2 + 0.0001*df_empleados$salario +
0.2*df_empleados$habilidades_sociales)
df_empleados$retencion <- factor(ifelse(runif(n) < prob, 1, 0))

head(df_empleados)
```

Interpretación Docente: El alumno debe notar que estamos creando un ecosistema de datos donde las variables tienen relaciones lógicas que el modelo debe "descubrir".

Paso 2: Regresión Lineal (Predecir Salario)

Dividimos los datos en entrenamiento (80%) y prueba (20%).

```
set.seed(123)
indice <- createDataPartition(df_empleados$salario, p = 0.8, list =
FALSE)
train_data <- df_empleados[indice, ]
test_data <- df_empleados[-indice, ]

# Entrenamiento del modelo
modelo_salario <- lm(salario ~ experiencia + certificaciones, data =
train_data)
summary(modelo_salario)
```

Interpretación Docente: Aquí el alumno analiza los coeficientes. ¿Cuánto aumenta el salario por cada año de experiencia extra? (\$5000 aprox. en este caso).

Paso 3: Clasificación KNN (Predecir Retención)

Usaremos KNN para clasificar si un empleado se queda o se va. ¡Recordar que KNN requiere escalamiento!

```
# Caret escala automáticamente con preProcess = c("center", "scale")
modelo_knn <- train(retencion ~ experiencia + habilidades_sociales +
salario,
                    data = train_data,
                    method = "knn",
                    preProcess = c("center", "scale"),
                    tuneLength = 5)

print(modelo_knn)
```

Interpretación Docente: El estudiante debe observar cómo cambia la precisión (Accuracy) según el valor de k (número de vecinos).

Paso 4: Clustering No Supervisado (K-Means)

Agruparemos a los empleados basándonos solo en su experiencia y salario.

```
datos_clustering <- df_empleados %>% select(experiencia, salario) %>%
scale()
set.seed(456)
clusters <- kmeans(datos_clustering, centers = 3)

# Visualización
df_empleados$cluster <- as.factor(clusters$cluster)
ggplot(df_empleados, aes(x = experiencia, y = salario, color = cluster))
+
  geom_point() +
  labs(title = "Segmentación de Empleados por Perfil") +
  theme_minimal()
```

Interpretación Docente: El alumno identifica 3 grupos: Junior, Mid y Senior. Esto demuestra cómo K-Means encuentra estructuras sin que le digamos qué buscar.

```
# =====
# PRÁCTICA 7: IMPLEMENTACIÓN DE MODELOS
# DE MACHINE LEARNING
# =====

# Librerías

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.metrics import accuracy_score

# Configuración

np.random.seed(789)

# =====
# PASO 1: GENERACIÓN DE DATOS
# =====

n=500

df_empleados=pd.DataFrame({

'experiencia':np.random.normal(5,2,n),

'certificaciones':np.random.poisson(3,n),

'habilidades_sociales':np.random.uniform(1,10,n),

'remoto':np.random.choice([0,1],n)

})

# Variable objetivo salario

df_empleados['salario']=(
25000
+
(df_empleados['experiencia']*5000)
+
(df_empleados['certificaciones']*2000)
+
np.random.normal(0,3000,n)
)

# Variable objetivo retención

```

```

prob=1/(1+np.exp(
-2
+
(0.0001*df_empleados['salario'])
+
(0.2*df_empleados['habilidades_sociales'])
))

df_empleados['retencion']=np.where(
np.random.rand(n)<prob,
1,
0
)

print(df_empleados.head())

# =====
# PASO 2: REGRESIÓN LINEAL
# =====

X=df_empleados[
['experiencia','certificaciones']
]

y=df_empleados['salario']

X_train,X_test,y_train,y_test=(
train_test_split(
X,
y,
test_size=0.2,
random_state=123
)
)

modelo_salario=LinearRegression()

modelo_salario.fit(
X_train,
y_train
)

```

```

predicciones=modelo_salario.predict(
X_test
)

print("\nCoeficientes")

print(
modelo_salario.coef_
)

print(
"\nIntercepto"
)

print(
modelo_salario.intercept_
)

rmse=np.sqrt(
mean_squared_error(
y_test,
predicciones
)
)

print("\nRMSE")

print(rmse)

# =====
# PASO 3: CLASIFICACIÓN KNN
# =====

X=df_empleados[
[
'experiencia',
'habilidades_sociales',
'salario'
]
]

y=df_empleados['retencion']

X_train,X_test,y_train,y_test=(

```

```
train_test_split(  
X,  
y,  
test_size=0.2,  
random_state=123  
)  
)  
  
# Escalamiento  
  
escalador=StandardScaler()  
  
X_train=escalador.fit_transform(  
X_train  
)  
  
X_test=escalador.transform(  
X_test  
)  
  
modelo_knn=KNeighborsClassifier(  
n_neighbors=5  
)  
  
modelo_knn.fit(  
X_train,  
y_train  
)  
  
pred=(  
modelo_knn.predict(  
X_test  
)  
)  
  
precision=accuracy_score(  
y_test,  
pred  
)  
  
print(  
"\nPrecisión KNN:"  
)  
  
print(  

```

```

precision
)

# =====
# PASO 4: K-MEANS
# =====

datos_cluster=df_empleados[
[
'experiencia',
'salario'
]
]

datos_cluster=escalador.fit_transform(
datos_cluster
)

kmeans=KMeans(
n_clusters=3,
random_state=456
)

clusters=kmeans.fit_predict(
datos_cluster
)

df_empleados['cluster']=clusters

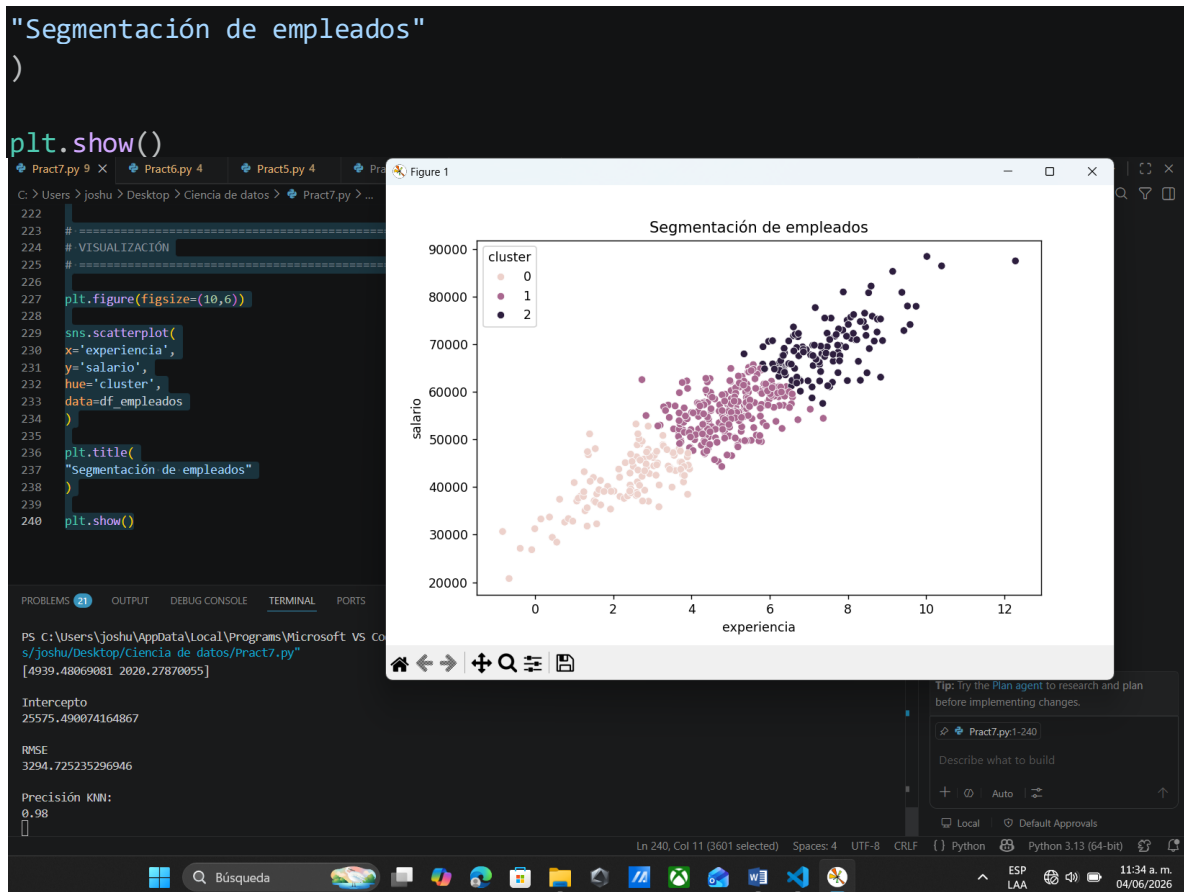
# =====
# VISUALIZACIÓN
# =====

plt.figure(figsize=(10,6))

sns.scatterplot(
x='experiencia',
y='salario',
hue='cluster',
data=df_empleados
)

plt.title(

```



Práctica 8: Clasificación de Intrusiones con KNN y Logística

Unidad 3: Modelado - TecNM

```
knitr::opts_chunk.set(echo = TRUE)
library(tidyverse)
library(caret)
```

Contexto Profesional

Un centro de datos del TecNM registra métricas de acceso. Tu tarea es clasificar si un acceso es **Seguro (0)** o **Ataque (1)** basándote en la latencia, el número de intentos fallidos y el tamaño del paquete.

Paso 1: Simulación de Tráfico de Red

```
set.seed(444)
n <- 400
df_red <- data.frame(
  latencia = rnorm(n, 20, 5),
  intentos_fallidos = rpois(n, 1),
  tamano_paquete = rnorm(n, 500, 100)
)

# Lógica de clasificación: Ataque si intentos > 3 o latencia > 40
df_red$es_ataque <- factor(ifelse(df_red$intentos_fallidos > 2 |
df_red$latencia > 35, 1, 0))
```

Paso 2: Modelo de Regresión Logística

Entrenamos para obtener la probabilidad de que una conexión sea un ataque.

```
modelo_log <- glm(es_ataque ~ ., data = df_red, family = "binomial")
summary(modelo_log)
```

Interpretación Docente: El alumno debe observar el valor de z y el p-value. Si el p-value de intentos_fallidos es muy pequeño (< 0.05), significa que esta variable es un predictor crítico para detectar ataques.

Paso 3: Clasificación con KNN

Implementamos KNN para ver cómo los "vecinos" (conexiones similares) ayudan a clasificar.

```
control <- trainControl(method = "cv", number = 5)
modelo_knn <- train(es_ataque ~ ., data = df_red, method = "knn",
  preProcess = c("center", "scale"),
  trControl = control, tuneLength = 10)
plot(modelo_knn)
```

Interpretación Docente: El gráfico muestra la precisión (Accuracy) según el número de vecinos (k). El alumno debe identificar el k **óptimo**. Un k muy bajo puede sobreajustar (capturar ruido), mientras que uno muy alto puede suavizar demasiado el modelo y perder ataques reales.

Paso 4: Análisis de Resultados (Clustering K-Means)

Finalmente, agruparemos las conexiones sin usar la etiqueta de ataque para ver si el algoritmo "descubre" el comportamiento anómalo por sí solo.

```
datos_clus <- df_red %>% select(-es_ataque) %>% scale()
set.seed(111)
```



```
clusters <- kmeans(datos_clus, centers = 2)
```

```
# Comparación: ¿Qué tan bien coincidió el cluster con la realidad?  
table(df_red$es_ataque, clusters$cluster)
```

Interpretación Docente: Si el cluster 1 contiene mayoritariamente "Ataques" y el cluster 2 "Seguros", el alumno concluye que los ataques tienen una firma digital tan distinta que son detectables incluso sin supervisión humana.

```
# =====  
# PRÁCTICA 8: CLASIFICACIÓN DE INTRUSIONES  
# CON KNN Y REGRESIÓN LOGÍSTICA  
# =====  
  
# Librerías  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.cluster import KMeans  
from sklearn.preprocessing import StandardScaler  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import accuracy_score  
from sklearn.metrics import confusion_matrix  
from sklearn.metrics import classification_report  
  
# Configuración  
  
np.random.seed(444)  
  
# =====  
# PASO 1: SIMULACIÓN DE TRÁFICO DE RED  
# =====  
  
n=400  
  
df_red=pd.DataFrame({  
  
    'latencia':np.random.normal(  
20,5,n  
)  
},
```

```

'intentos_fallidos':np.random.poisson(
1,n
),

'tamano_paquete':np.random.normal(
500,100,n
)

})

# Definir ataques

df_red['es_ataque']=np.where(

(df_red['intentos_fallidos']>2)

|

(df_red['latencia']>35),

1,

0

)

print(df_red.head())

# =====
# PASO 2: REGRESIÓN LOGÍSTICA
# =====

X=df_red.drop(
'es_ataque',
axis=1
)

y=df_red['es_ataque']

X_train,X_test,y_train,y_test=(

train_test_split(

X,

```

```
y,  
test_size=0.2,  
random_state=123  
  
)  
  
)  
  
modelo_log=LogisticRegression()  
  
modelo_log.fit(  
X_train,  
y_train  
)  
  
pred_log=(  
modelo_log.predict(  
X_test  
)  
)  
  
print("\nPrecisión Regresión Logística:")  
  
print(  
accuracy_score(  
y_test,  
pred_log  
)  
)  
  
print("\nMatriz de confusión")  
  
print(  
  
confusion_matrix(  
y_test,  
pred_log  
)  
  
)  
  
print("\nReporte")  
  
print(  

```

```

classification_report(
y_test,
pred_log
)

)

# =====
# PASO 3: KNN
# =====

escalador=StandardScaler()

X_train=escalador.fit_transform(
X_train
)

X_test=escalador.transform(
X_test
)

accuracy=[]

for k in range(1,11):

    modelo_knn=KNeighborsClassifier(
        n_neighbors=k
    )

    modelo_knn.fit(
        X_train,
        y_train
    )

    pred=(
        modelo_knn.predict(
            X_test
        )
    )

    accuracy.append(

        accuracy_score(
            y_test,

```

```

        pred
    )

)

# Mejor K

mejor_k=np.argmax(
accuracy
)+1

print(
"\nMejor valor K:"
)

print(
mejor_k
)

# =====
# GRÁFICA DE ACCURACY
# =====

plt.figure(figsize=(8,5))

plt.plot(
range(1,11),
accuracy
)

plt.xlabel(
'Número de vecinos'
)

plt.ylabel(
'Accuracy'
)

plt.title(
'Precisión según K'
)

plt.show()

```

```

# =====
# PASO 4: CLUSTERING K-MEANS
# =====

datos_cluster=df_red.drop(
    'es_ataque',
    axis=1
)

datos_cluster=escalador.fit_transform(
    datos_cluster
)

kmeans=KMeans(
    n_clusters=2,
    random_state=111
)

clusters=kmeans.fit_predict(
    datos_cluster
)

df_red['cluster']=clusters

print(
    "\nComparación:"
)

print(
    pd.crosstab(
        df_red['es_ataque'],
        df_red['cluster']
    )
)

# =====
# VISUALIZACIÓN
# =====

plt.figure(figsize=(10,6))

sns.scatterplot(

```

```
x='latencia',  
y='intentos_fallidos',  
hue='cluster',  
data=df_red  
)  
  
plt.title(  
'Clusters de conexiones'  
)  
  
plt.show()
```

